Python Fundamentals

Part 1 — Concepts

Output / Input · Variables · Data Types · Operators

1. Output

The first and easiest thing we can do in Python is printing (or outputting) something on the screen. We perform this simple task using a special function called the print() function.

What exactly is a function? We'll learn about it in the next chapter, but for now we can imagine it to be a specialized helper that performs a specific task. So, print() is our little helper that helps us display something on the screen (the console). We can use it to show information, results of calculations, or even format our text.

print() Syntax

```
print("hello world")   # output will be: hello world
```

Anything inside single quotes (' ') or double quotes (" ") is called a string and gets printed as it is. We can also print numerical values:

```
print(3.14)      # output: 3.14
print(2 + 3)     # output: 5
```

Each print statement displays output on a different line, but we can also display multiple values on the same line:

```
print("hello world", "from PRIME")
```

2. Python Character Set

A character set is a collection of characters, numbers, and symbols that our language recognizes and we can use. Following are the common characters we'll be frequently using in Python:

1. English letters — `A to Z` and `a to z`
2. Digits — `0 to 9`
3. Special symbols — `+  -  *  /  %  ,  etc.`
4. Whitespaces & Escape characters — `blank space,  \t  (tab),  \n  (newline), etc.`
5. All ASCII & Unicode characters as part of data & literals

3. Variables

A variable is like a container that stores data or values in our program. We can think of it as a labeled box that we can put something in, change it, or use it later.

Variables in Python:

1. Have names called identifiers.
2. Can store data of any type (numbers, text, etc.).

Some examples:

```
name = "Shradha"
age = 30
```

```
PI = 3.14
word = "artificial intelligence"
print("value of PI =", PI)
```

Variable Naming (Identifier) Rules

1. Name must start with a letter (a–z, A–Z) or underscore (_).
2. Can contain letters, numbers, or underscores after the first character.
3. Cannot use Python keywords (like `if`, `for`, `class`) as variable names.

Invalid variable names:

```
2name = "Bob"    # Starts with a number – INVALID
class = 10       # 'class' is a keyword – INVALID
```

Important Notes

1. **Python is dynamically typed.** When creating variables we don't need to declare the type explicitly.
2. **Python is case-sensitive.** `age` and `Age` are different variables.
3. **Indentation is important.** The general standard is 4 spaces per indentation level.
4. **Keywords are reserved words** in Python and their meaning is pre-defined. Examples: `if, for, class, while, else, in, def`

4. Data Types

A data type defines the kind of value a variable can hold. Python has several built-in data types. Let's look at the simplest ones to start with:

Type	Description	Example
`int`	Integer values (whole numbers)	`10, -5, 0`
`float`	Floating-point values (decimals)	`3.14, -0.5`
`str`	Strings (sequence of characters)	`"hello", 'world'`
`bool`	Boolean values	`True, False`
`None`	Represents absence of a value	`None`

We also have other data types like `complex`, `list`, `tuple`, `set`, etc. that we'll cover in coming chapters.

To check the variable type we can use the type() function:

```
x = 10
print(type(x))           # <class 'int'>

PI = 3.14
print(type(PI))          # <class 'float'>

name = "shradha"
print(type(name))        # <class 'str'>

isTeacher = True
print(type(isTeacher))   # <class 'bool'>

empty_var = None
print(type(empty_var))   # <class 'NoneType'>
```

Type Conversion & Type Casting

Type conversion is when we convert (cast) variables from one type to another. It can happen in 2 ways:

1. Type Conversion (Implicit)

Done automatically by Python:

```
a = 5
b = 3.0
print(a + b)    # Python converts result to float by default → 8.0
```

2. Type Casting (Explicit)

Done by the programmer using conversion functions:

```
x = 10
y = float(x)    # convert to float → 10.0
z = str(x)      # convert to string → "10"
```

Common type conversion functions: `int()`, `float()`, `str()`, `bool()`, `list()`, `tuple()`

5. Operators

An operator is a symbol that tells Python to perform a specific computation on one or more values (called operands).

```
print(1 + 3)    # '+' is an Operator & (1, 3) are operands
```

There are several types of operators in Python. Let's start by understanding some of the most common ones:

5.1 Arithmetic Operators

Used to perform math operations.

Operator	Name	Example	Result
+	Addition	a + b	15
-	Subtraction	a - b	-5
*	Multiplication	a * b	50
/	Division	a / b	0.5
%	Modulo (remainder)	a % b	5
**	Power / Exponent	a ** b	9765625

Example (a = 5, b = 10):

```
a = 5
b = 10
print(a + b)     # Addition → 15
print(a - b)     # Subtraction → -5
print(a * b)     # Multiplication → 50
print(a / b)     # Division → 0.5
print(a % b)     # Modulo → 5
print(a ** b)    # Power → 9765625
```

5.2 Relational Operators

Used to compare values. They return True or False.

Operator	Meaning	Example (a=10, b=20)
==	Equal to	`a == b → False`
!=	Not equal to	`a != b → True`
>	Greater than	`a > b → False`
<	Less than	`a < b → True`
>=	Greater than or equal to	`a >= b → False`
<=	Less than or equal to	`a <= b → True`

5.3 Assignment Operators

Used to assign values to variables.

```
x = 10              # Simple assignment → x = 10
x += 5              # equivalent to x = x + 5 → 15
x -= 3              # equivalent to x = x - 3 → 12
x *= 2              # equivalent to x = x * 2 → 24
x /= 4              # equivalent to x = x / 4 → 6.0
x %= 4              # equivalent to x = x % 4 → 2.0
x **= 3             # equivalent to x = x ** 3 → 8.0
```

5.4 Logical Operators

Used to combine boolean values.

val1	val2	val1 and val2	val1 or val2	not val1	not val2
T	T	T	T	F	F
T	F	F	T	F	T
F	T	F	T	T	F
F	F	F	F	T	T

Example (x = 10, y = 20, z = 5):

```
x = 10
y = 20
z = 5
print(x > z and y > x)    # True  (AND)
print(x > y or y > z)     # True  (OR)
print(not (x > y))        # True  (NOT)
```

We'll cover more operator types like membership operators in future chapters.

Operator Precedence

Operators have a priority — there is a specific order in which operations are performed when we have multiple operators in the same expression. The order is as follows (highest priority first):

- **`()`** — Parentheses (highest priority)
- **`**`** — Exponent
- **`+x, -x, ~x, not`** — Unary operators
- **`*, /, //, %`** — Multiplication, division, floor division, modulus
- **`+, -`** — Addition, subtraction
- **`<<, >>`** — Bitwise shifts
- **`&`** — Bitwise AND
- **`^`** — Bitwise XOR

- `|` — Bitwise OR
- `<, <=, >, >=, !=, ==` — Comparison operators
- `not` — Logical NOT
- `and` — Logical AND
- `or` — Logical OR
- `=, +=, -=, ...` — Assignment operators

> **Note — Unary Operators**
> Unary operators work on a single operand (a single value or variable). Examples: +x (unary plus), -x (unary minus), not. We don't generally use +x as numbers are positive by default.

6. Input

We use the input() function to take input from the user while the program is running. Whatever the user types at the input prompt is returned as a string.

Syntax:

```
name = input("enter name: ")
print(name)
print(type(name))    # type is always string
```

We can use type conversion functions to convert the type of input:

```
# Program to add 2 numbers entered by the user
a = int(input("enter 1st num: "))
b = int(input("enter 2nd num: "))
print(a + b)
```

7. Practice Problem

Let's summarize all the concepts we learnt in this chapter into a simple practice problem.

Write a program that takes in 2 numbers (a & b) and prints the average.

```
a = int(input("enter 1st num: "))   # e.g. 5
b = int(input("enter 2nd num: "))   # e.g. 10
avg = (a + b) / 2
print("average =", avg)            # → 7.5
```

Keep Learning & Keep Exploring!